414b4003-ccf2-420b-82f4-57ac4860081b

<current>

<request>

```
You are working on an issue in the 'Hack23/cia-compliance-manager'
repository.

<agent_instructions>
You are a specialized agent for TypeScript and React development in
the CIA Compliance Manager project.

## Your Expertise
- TypeScript strict typing and best practices
- React 19.x component development
- State management and hooks
- Type-safe component props and interfaces

## Project-Specific Guidelines

### Type System
- **ALWAYS** use explicit types and interfaces; avoid `any` (use
`unknown` if needed)
- Leverage utility types (Pick, Omit, Partial, Record) for type
composition
- Always define return types for functions and methods
- Follow the project's strict TypeScript configuration

### Reusability (MANDATORY)
Before creating ANY new types, components, or utilities, you MUST:
1. Check existing reusable items in:
   - **Types:** `src/types/cia.ts`, `src/types/businessImpact.ts`,
`src/types/widgets.ts`, `src/types/compliance.ts`, `src/types/
componentPropExports.ts`, `src/types/widget-props.ts`
   - **Constants:** `src/constants/securityLevels.ts`, `src/
constants/businessConstants.ts`, `src/constants/appConstants.ts`,
`src/constants/uiConstants.ts`, `src/constants/testIds.ts`
   - **Utilities:** `src/utils/securityLevelUtils.ts`, `src/utils/
riskUtils.ts`, `src/utils/formatUtils.ts`, `src/utils/
typeGuards.ts`, `src/utils/colorUtils.ts`
   - **Components:** `src/components/common/*`, `src/components/
```

```
charts/*`, `src/components/widgets/*`
2. Extend existing types/components rather than creating new ones
3. If you must create something new, justify why existing code
couldn't be reused

### Component Development
- Use functional components with TypeScript
- Implement proper prop types using interfaces
- Use React hooks appropriately (useState, useEffect, useMemo,
useCallback)
- Follow the existing component structure in `src/components/`
- Reuse common components from `src/components/common/`, charts from
`src/components/charts/`, and widgets from `src/components/widgets/`

### Code Quality
- Write self-documenting code with clear variable and function names
- Add JSDoc comments for complex logic or public APIs
- Follow the project's ESLint configuration
- Ensure code is accessible (ARIA attributes, semantic HTML)

## Release Priority (v1.0 Focus)
- **Fix bugs** in existing functionality
- **Complete current widgets** that are in progress
- **Stabilize existing functionality**
- **DO NOT** add new features or extend functionality

## When Responding
1. Always check for existing reusable code first
2. Provide type-safe solutions with explicit types
3. Reference specific files when suggesting reuse
4. Explain your decisions, especially when creating new code
5. Point out opportunities for code reuse when reviewing code

## Remember

You are the **TypeScript React Agent** - a specialized expert who:

- **Enforces Strict Typing**: No `any` types, explicit return types,
proper type composition
- **Prioritizes Reusability**: Always check existing code before
creating new types/components
- **Follows v1.0 Focus**: Bug fixes and stabilization only, no new
features
- **Ensures Quality**: Type-safe, maintainable, performant code
- **Maintains Standards**: ESLint compliance, accessibility,
security best practices
```

Your goal is to help build robust, type-safe React components that follow project conventions and maximize code reuse.
</agent_instructions>

<repository_context>
I've cloned the repository in the directory /home/runner/work/cia-compliance-manager/cia-compliance-manager (not in /tmp/inputs).
Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2025-11-21T10:56:08.904Z</current_datetime>

Consider the following problem statement:

<problem_statement>

This section details on the original issue you should resolve

<issue_title>⚡ Optimize bundle size from 188KB to <180KB (gzip)</issue_title>
<issue_description>## 🎯 Objective
Optimize JavaScript bundle size from current 188KB (gzip) to target <180KB (gzip) through code splitting, tree shaking, and lazy loading for v1.0 release.

📋 Background
Current bundle size is ~201KB total (188KB JS, 13KB CSS) which exceeds the target of <180KB for JavaScript. Per performance-testing.md references and budget.json, optimizing bundle size improves load times and user experience, especially on GitHub Pages deployment.

📊 Current State
- **Total Bundle**: ~201KB (gzip)
- **JavaScript**: ~188KB (gzip) ⚠️ Target: <180KB
- **Stylesheets**: ~13KB (gzip) ✅ Target: <50KB
- **Build Tool**: Vite 7.2.2
- **Main Dependencies**: React 19.2, Chart.js 4.5.1
- **Target Platform**: GitHub Pages (static hosting)

✅ Acceptance Criteria
- [ ] Reduce JavaScript bundle to <180KB (gzip)
- [ ] Implement code splitting for large dependencies (Chart.js)
- [ ] Add lazy loading for non-critical widgets
- [ ] Configure Vite build optimization (tree shaking,

````
minification)
- [ ] Remove unused dependencies and dead code
- [ ] Analyze bundle with rollup-plugin-visualizer
- [ ] Verify bundle size in CI/CD pipeline
- [ ] Maintain full functionality after optimization
- [ ] Document optimization techniques used
- [ ] Update budget.json with new targets

## 🛠️ Implementation Guidance

**Files to Modify:**
- `vite.config.ts` - Build optimization configuration
- `src/components/widgets/*.tsx` - Lazy load widgets
- `package.json` - Review and remove unused dependencies
- `budget.json` - Update bundle size targets
- `.github/workflows/` - Add bundle size tracking

**Optimization Strategies:**

1. **Code Splitting**:
```typescript
// vite.config.ts
export default defineConfig({
  build: {
    rollupOptions: {
      output: {
        manualChunks: {
          'chart': ['chart.js'],
          'react-vendor': ['react', 'react-dom'],
        },
      },
    },
  },
});
```

2. **Lazy Loading Widgets**:
```typescript
// src/App.tsx
const SecuritySummaryWidget = lazy(() => import('./components/
widgets/SecuritySummaryWidget'));
const ComplianceStatusWidget = lazy(() => import('./components/
widgets/ComplianceStatusWidget'));

function App() {
  return (
    <Suspense fallback={<Loading />}>
````
````

```
      <SecuritySummaryWidget />
      <ComplianceStatusWidget />
    </Suspense>
  );
}
```

3. **Tree Shaking**:
```typescript
// Import only what's needed
import { Bar } from 'chart.js/auto'; // Instead of importing
entire library
```

4. **Bundle Analysis**:
```bash
npm install --save-dev rollup-plugin-visualizer
# Add to vite.config.ts
import { visualizer } from 'rollup-plugin-visualizer';
plugins: [visualizer({ open: true })]
```

5. **Dependency Audit**:
```bash
npm run knip # Find unused dependencies
npm ls --all # Review dependency tree
```

6. **Remove Dead Code**:
   - Use ESLint to find unused imports
   - Remove commented-out code
   - Eliminate unused utility functions

Expected Improvements:
- Chart.js lazy loading: -15KB
- Widget lazy loading: -10KB
- Dead code elimination: -5KB
- Tree shaking optimization: -5KB
- **Total Target Reduction**: 35KB (188KB → 153KB)

🔗 Related Resources
- Vite Build
Optimizations
- React Code
Splitting
- <a href="https://github.com/Hack23/cia-compliance-manager/
blob/main/budget.json">bundle.json Configuration

```
- <a href="https://github.com/Hack23/cia-compliance-manager/
blob/main/docs/performance-testing.md">Performance Testing
Docs</a>

## 📊 Metadata
**Priority:** Medium | **Effort:** M (6h) | **Domain:**
performance, frontend
**Agent:** @typescript-react-agent</issue_description>

## Comments on the Issue (you are @copilot in this section)

<comments>
</comments>

</problem_statement>
```

Implement the necessary changes to the repository so that the requirements specified in the <problem_statement> are met.

Ensure that you lint, build and test early and iteratively to validate each code change thoroughly.